

Detecting Aimbot Usage in Minecraft Using a Long Short-Term Memory Model

James Vandersluis^{†,*}, Alexander Wong[†]

[†] University of Waterloo

Abstract

The gaming industry has long been at the cutting edge of technology, sometimes at the cost of proper game security. This has led to an industry of cheaters, in particular among online video games. The ability to catch cheaters is of interest to game server operators who want to maintain integrity and trust with their player-base. A form of cheating that is particularly problematic is the use of aiming robots (aimbots) that use client-side hacks to lock onto targets and perform maximum damage. Aimbots are common in first-person style player-vs-player type video games where they have the most potential to give the cheater a significant unfair advantage. Minecraft, for example, is a game that has a large online player base who enjoy player-vs-player combat mini-games. Hacked clients are readily available to the average player and thus there is a strong need to be able to detect this cheating behaviour on the server side. Significant consideration toward fairness must be given to any solution that attempts to detect behaviour that would result in disciplinary action. Thus, it is important to maintain very high levels of accuracy as to avoid the false flagging of players who did not cheat. Further, any system that does not detect true aimbot users with very high accuracy would undermine player trust in the system. Therefore, we propose in this paper, a neural network approach to aimbot detection using Long Short-Term Memory to classify cheaters based on their movements over time. Our network achieves very high accuracy at high confidence levels, providing a notable improvement over a Gated Recurrent Network on the same data. We believe the network at these accuracy levels could have significant practical use in real-world servers.

Keywords: Cheat Detection, Machine Learning, Online Gaming, Aimbotting, LSTM

1. Introduction

Historically the video game industry has stayed at the cutting edge of technology, sometimes at the cost of implementing proper security features [1]. This gives many would-be hackers an easier time when trying to find exploits in games. This is a smaller problem in single player games since the exploits would not affect other players directly. However, in the online gaming sphere the poor security of a game may allow hackers to leverage exploits to their own advantage. Though there are a wide range of exploit categories, one of particular interest is cheating through misplaced trust. This is the act of tampering on the client-side with the game code, configuration data, or both, allowing one to take advantage of the trust put in the client [2]. A cheater can tamper with their client's game states directly in a way that would not otherwise be possible for players [2]. A specific example of this kind of exploit would be the automatic aiming robot, also known as an aimbot. Aimbots use the internal game state of the client to lock the player's camera perfectly onto the target without any human intervention.

This specific form of exploitation is seen often in first-person combat games, where the accuracy of one's mouse movements and click timings is critical. One such video game that suffers from this form of cheating is Minecraft. Minecraft is an incredibly popular game, having been purchased over 300 million times [3]. In the game of Minecraft, players control a character from a first-person perspective in a voxel based world. The game consists of a

*james.vandersluis@uwaterloo.ca



Figure 1. A screenshot of Minecraft showcasing the first-person nature of the game.

procedurally generated map in which players can place and break blocks, gather resources, craft items or blocks, fight with monsters, or fight with other players. Many players enjoy playing the game in its multiplayer mode. Multiplayer experiences come in the form of individual servers that are hosted by third-parties. Some of these servers have massive player bases. For example, Hypixel is an incredibly popular server that garners tens of thousands of unique players online per day [4]. The server hosts many player-vs-player (PvP) based competitive minigames in which different players fight each other to win.

There is a lot of skill involved in PvP combat. Players must be strategic, have fast reflexes, maintain accuracy, and perform well-timed mouse clicks. As many players compete to climb server leaderboards, this leads some to pursue in-game cheating to gain a competitive advantage. A community of cheaters has risen in the Minecraft scene, creating open source clients such as LiquidBounce [5] to provide a whole host of cheating utilities to the masses. One of the offerings of the LiquidBounce client is an aimbot that locks onto targets and simulates mouse clicks that are perfectly timed to inflict maximum damage in combat. It is clear to see that a third-party who is hosting a server would have an incentive to reduce cheating in their servers to boost player retention rates. Thus, the highly accurate detection of such aimbots is important to maintain player trust.

Our paper starts with a background of different approaches to the aimbot detection problem. Next, we introduce a neural network approach to aimbot detection, utilizing Long Short-Term Memory (LSTM) as the primary architecture block to identify aimbot patterns from movement data. Then taking inspiration from literature [6], we adopted their proposed Gated Recurrent Unit (GRU) architecture, tailoring it to our use case, allowing us to demonstrate the benefits of LSTM on long-sequence data. We then discuss our results and compare them to the performance of similar aimbot detectors. We also investigate the viability of our model in a live setting. We conclude our paper with a summary of our present work along with a list of potential future works we identified.

In summary, our paper makes the following contributions,

- An LSTM-based neural network capable of detecting aimbot usage in Minecraft with very high accuracy at high confidence levels, proving to be a promising solution that keeps the emphasis on fairness.
- An exploration of the feasibility of the proposed network on real game servers.

2. Background

2.1. Statistical Approaches

Though neural networks are a good choice for a pattern recognition task like aimbot detection, there are other methods that rely on more statistical measures. One such paper explored the use of two dissimilarity metrics to distinguish whether a player’s movement trajectory was indicative of a bot or a human [7]. Their approach involved computing the dissimilarity metric, applying an isomap to the data, and then using a classification algorithm that could identify the clusters such as k-NN or smooth SVM [7]. Their game of choice was Quake 2, a first-person shooter game that had a large amount of player aiming data available [7]. With this dataset they accomplished an accuracy score of 98% and higher [7].

Another interesting paper attempts to identify aimbots by examining whether a player’s performance is consistent in areas that cannot be performed by an aimbot [8]. This approach exploits the fact that players who cheat will have expert level aim, but often resemble the average players in all other aspects of the game [8]. The authors specifically focus on analyzing how players kill their opponents and how players themselves get killed in the game Counter Strike Global Offensive (CS:GO) and Counter Strike-1.6 (CS-1.6) [8]. The authors created server-side software that implemented their statistical techniques to efficiently perform real-time detection, detecting 32 out of 35 aimbotters in CS-1.6 and 7 out of 8 aimbotters in CS:GO [8].

2.2. Deep Learning Approaches

To the best of our knowledge, leveraging LSTMs for tackling the problem of server-side detection of aimbot usage in Minecraft has not been previously explored. Furthermore, the notion of leveraging LSTMs for aimbot usage detection in general has not been well-explored outside of [9, 10]. In [9], the author introduced the use of LSTM to detect aimbotting in CS:GO [9]. The author scraped data from real game-play to construct time series samples for their network; some of their features include: *x, y, and z positions, the velocities in the x, y, and z directions, pitch, yaw, kills, and deaths* along with other game-play specific features [9]. Their feature set draws some parallels to the dataset we are using for our approach with respect to the positions, pitch, and yaw. The author performed ten-fold cross validation to evaluate their model, yielding test set accuracy in the range of 80-90% [9].

Another author explored the use of LSTMs to detect aimbotting, looking also at the game CS:GO [10]. This study used a relatively small amount of data, however, they still saw decent performance, at 70% accuracy [10]. The author notes however, that in order for their model to be useful it would need to be at least 90% accurate, which would require much more training data [10]. While the aforementioned studies illustrate potential for LSTMs as mechanisms worth exploring for aimbot detection, the ability to leverage LSTMs to tackle the problem of server-side detection of aimbot usage in Minecraft requires special design considerations, which we will detail in later sections.

A different study [6], investigated a Recurrent Neural Network (RNN) approach to aimbot detection in the popular online first-person shooter video game CS:GO, and saw strong results on their test set with 97.64% accuracy. This study hired a variety of players ranging from average to expert skill levels to collect data on game-play [6]. Collected data included the players’ positions and aiming angles, which were used to train their bidirectional Stacked Gated Recurrent Unit, which they claim had similar performance to an LSTM [6]. In this paper the authors trained an RNN on benign player aiming data, such that the trained RNN would produce an anomaly score at inference time for the input data [6]. The idea was that since the RNN was only trained on benign movements, any movements from an

aimbot would provide worse RNN predictions (higher anomaly score) as it deviates from the behaviour of the benign movements [6].

One other approach to detect aimbotting and trigger-botting was explored by the authors of [11], where they used mouse positions, mouse events, and keyboard events to detect cheating behaviour via a feed-forward convolutional neural network. This paper saw strong performance with 99.2% and 98.9% accuracy for their trigger-bot and aimbot, respectively [11]. In our approach however, the model does not have access to these features specifically, as the servers do not collect the raw window event data but rather the game state information as we discuss further in Section 5.2.

3. Methodology

3.1. Dataset Preparation

The dataset used for our study comes from Kaggle and contains two large tables of time series data [12]. One table is the raw aiming information captured from a Minecraft user who does not have an aimbot enabled and the other table is the raw aiming information from a user who does have an aimbot enabled [12]. The author of the dataset calls these the legit and cheater tables, respectively [12]. The data is sorted by groups of unique IDs that appear to correspond to different sessions, as the same unique IDs are present in both the cheater and legit tables. Within each table we have access to fifteen features. The **yaw**, **pitch**, **delta_yaw**, **delta_pitch**, **accel_yaw**, **accel_pitch** features describe the player’s camera rotations and their changes between samples. The **target_x**, **target_y**, **target_z** features indicate where the player’s camera is looking. The **position_x**, **position_y**, and **position_z** features specify the player’s coordinates. The **sensitivity** feature is the player’s camera sensitivity. The **time** feature in the milliseconds since Unix Epoch format captures when the sample was taken. Lastly, the **new_sequence** feature is set to true when a new movement sequence has started, and is false when the sample is within a sequence.

In many other first-person combat games, aiming behaviour may involve more nuanced details of what may serve to characterize a player of being in suspicion of aimbotting. For example, in a first-person shooter style game, usually more damage is dealt by hitting specific regions of the opponents’ hitbox. Thus, information about what areas are being hit may be critical in determining suspicious behaviour. Whereas in Minecraft, the location of a strike within combat is not relevant in the damage calculations. Rather, combat is generally very sporadic, involving significant clicking, and rapid mouse movements to try and track the player during their movements in close quarters. It is for this reason that we believe the features used for detection, being the movement, aiming, and positional information as detailed above, is sufficient to make accurate predictions about suspicious behaviour. As within a game like Minecraft, it would be highly unlikely that a player would exhibit any kind of lock-in-like behaviour on a target given the nature of its combat system.

The first step to processing this data was splitting the tables into groups by unique ID. We then sorted the samples by the **time** column to ensure all the readings were in correct chronological order. Next, we cut the dataset into individual samples. We did this by utilizing the **new_sequence** column in the dataset, splitting a new data point at each new sequence. Next, we removed the **sensitivity**, **time**, and **new_sequence** columns from the data row as we believed that these features were not significant to whether someone was aimbotting or not. The rationale for this is that the user’s sensitivity does not affect their targeting behaviour, the time is simply affected by the sampling rate, and the new sequence is purely a measure of when the user started their movement, which we already used for splitting the samples. We then appended these individual samples to our processed dataset array and the target class label being either 0 if it was a legit sample or 1 if it was a cheater sample. We believed it to be most appropriate to split on each **new sequence** since in

practice this would be a similar behaviour to what we imagined a server-side implementation would do.

The authors of BotScreen, an RNN based aimbot detector, noted that RNNs generally expect sequences to exist in regular intervals [6]. Thus, they normalized their data into 16ms intervals via linear interpolation of the event data [6]. We did not need to take this approach in our paper, as the sampling rate of the data was naturally very consistent.

As many of the features vary in scale, we performed z-score normalization. We found in our testing that this normalization resulted in better convergence during training when compared to no normalization.

3.2. Network Architecture

It is evident that by the nature of this problem being a time series, that some form of RNN was most applicable. We weighed the possible available options by consulting literature to explore approaches to similar problems. Common networks for time series data were Long Short-Term Memory and Gated Recurrent Unit networks as shown in Section 2. LSTM is a type of recurrent neural network that gives a solution to the vanishing/exploding gradient problem seen in traditional RNNs [13]. LSTM is known to be effective at capturing long-term temporal dependencies [14]. Further, they are very generalized and can handle many kinds of problems [14]. Gated Recurrent Unit networks are similar to LSTMs, however, they omit the output gate that controls the level to which its memory is exposed, and thus exposes the full state at each time step [15]. From empirical results in one study, it was not possible to determine whether LSTM or GRU was inherently a better kind of recurrent network [15]. Rather, it appeared as though the choice of which network performs better depends on the dataset [15]. Transformers were also not considered, since they are generally less suited towards time series data, especially when the series are long-term, as shown in a study where the authors compared Transformer models to LSTM models in a stock price time series task [16]. We hypothesized that an LSTM network would be best suited for the detection of aimbotting on our dataset’s features since the sequences are reasonably long and would thus benefit from the use of the output gate. To test our hypothesis we explored the use of both our own proposed LSTM network and a bidirectional GRU based network proposed in [6]. We slightly modified the model in [6] by switching the loss metric to binary cross entropy loss opposed to their use of mean squared error. This was done to support our desired binary outputs from the model. It follows therefore, that we also added a sigmoid activation on the output of the model. For the GRU based model we used all the other hyper-parameters detailed in [6], being, a 3 layer bidirectional GRU with 10% dropout, 64 hidden units per layer, and batch sizes of 64.

Our proposed network architecture shown in Figure 2 consists of a four-layered LSTM block followed by a fully connected layer that connects the hidden states in the final layer of the LSTM to a single output neuron. For the first three layers of the LSTM we applied 30% dropout to help improve regularization of the network. We applied the sigmoid function to bind our output to (0, 1), effectively giving us a probability of whether the input data is an aimbot or not. The input to the network is a sequence of 35 consecutive movement samples from the player where each sample consists of 12 features, being: yaw, pitch, delta_yaw, delta_pitch, accel_yaw, accel_pitch, target_x, target_y, target_z, position_x, position_y, and position_z. During training, validation, and testing we used batch sizes of 128.

3.3. Evaluation Metric

To evaluate both models’ performance we split our dataset into an 80/10/10 train, validation, and test set, where all data is shuffled prior to splitting to reduce correlation between

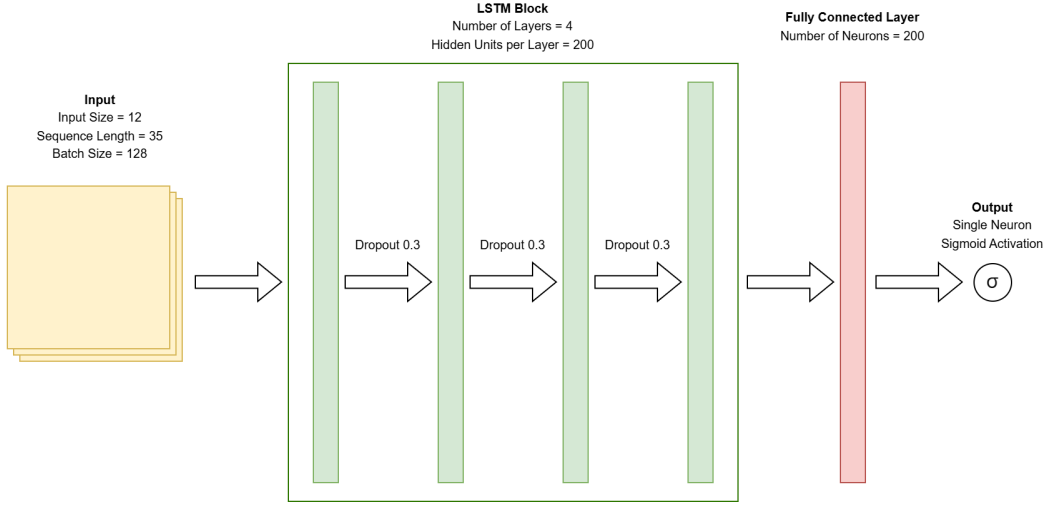


Figure 2. Our proposed architecture consisting of 4 LSTM layers with 30% dropout, a fully connected layer, and a sigmoid output neuron.

samples in the respective sets. The validation set is used to evaluate the model’s performance during training for purposes of fine-tuning the model and hyper-parameter searching. The test set is held out until the end of our study to evaluate the model on new data to see how well it generalizes.

The metric we use to evaluate on the validation and test set is the following definition of accuracy [17].

$$Accuracy = \frac{TruePositives + TrueNegatives}{TotalNumberOfSamples}$$

Where TruePositives are the number of samples correctly identified as aimbotting, and TrueNegatives are the number of samples correctly identified as non-aimbotting behaviour. Accuracy is a metric that evaluates which proportion of samples were identified correctly. For both binary classifier models, this is a meaningful metric as the output is either true or false with respect to whether the movement is indicative of aimbotting or not. Further, we care about accuracy being high, as false positives or false negatives means either a player is being flagged and potentially punished for an offense they did not commit, or the player committing the offense is not being caught at all. Thus, it is critical for the model to be usable in real-world applications in which the promotion of fairness is key, that it be able to maintain a high accuracy.

To better understand the confidence of the model, we used seven accuracy threshold values that determine at which confidence level the model classifies the prediction as a true positive. These seven values are: 0.5, 0.6, 0.7, 0.8, 0.9, 0.95, and 0.99. Where a high threshold represents a higher level of confidence required to classify the sample as aimbotting.

3.4. Training Strategy and Fine-Tuning

We implemented and trained two architectures for our study. We first trained the bidirectional GRU based network as described in Section 3.2 using the hyper-parameters as discussed. No further hyper-parameter exploration was done for this model.

Table 1. The range of hyper-parameters that we manually searched along with the parameters in our search that yielded the best results for our LSTM model.

PARAMETER	RANGE	BEST
SEQUENCE LENGTH	25 - 45	35
HIDDEN UNITS	75-300	200
NUMBER OF LAYERS	1-4	4
DROPOUT PERCENTAGE	0.25 - 0.5	0.3

Each model was trained for 64 epochs to match the number of epochs used in [6]. We used check-pointing during training to save each model’s weights at each epoch, keeping the weights that achieved the highest validation set accuracy for each model.

For our LSTM model we performed a manual search based on intuition and empirical results. We began tuning hyper-parameters in the ranges listed in Table 1. These ranges were chosen as they were in the neighbourhood of our initial good results. We found during our search that sequence lengths less than 25 tended to result in quickly degrading performance. We believe that sequences of this length were not sufficient to capture the pattern details required for proper classification. Further, we experimented with the network size and settled on the chosen best parameters, as they seemingly gave the network enough complexity to learn the detailed patterns in the time series, while also not being too complex as to hinder learning. It may be of interest as further work to perform a more rigorous hyper-parameter search that may yield better results than ours.

4. Experimental Results

Table 2. Classification accuracies for the test set using seven different confidence thresholds on the LSTM and GRU network, respectively.

CONFIDENCE LEVEL	LSTM			BIDIRECTIONAL GRU		
	CORRECT	INCORRECT	ACCURACY	CORRECT	INCORRECT	ACCURACY
0.50	33535	394	98.8%	33283	646	98.1%
0.60	33534	395	98.8%	33288	641	98.1%
0.70	33517	412	98.8%	33244	685	98.0%
0.80	33507	422	98.8%	33200	729	97.9%
0.90	33462	467	98.6%	33033	896	97.4%
0.95	33404	525	98.5%	32782	1147	96.6%
0.99	33122	807	97.6%	32048	1881	94.5%

For both the LSTM and bidirectional GRU based models, we used the best model as determined by the highest validation set accuracy, as shown in Figure 3, for use against the test set. We see in Figure 3 that the LSTM was able to surpass and saturate at a higher accuracy than the bidirectional GRU during training when evaluated on the validation set. Both the LSTM and the bidirectional GRU managed to achieve very strong results on the held out test set. Table 2 outlines the results from the seven evaluation confidence levels. We see in this table that in the confidence levels of 0.5–0.9 the two architectures yield similar accuracies. However, at the very high confidence levels the LSTM starts to show noticeable improvements over the bidirectional GRU. We see this divergence appear at the 0.95 and 0.99 confidence level specifically. The LSTM was able to detect aimbotters with 98.5% and 97.6% accuracy at the 0.95 and 0.99 confidence levels, respectively. Meaning, for example, where the model was 99% (0.99) confident, it identified 97.6% of the aimbotters. This is

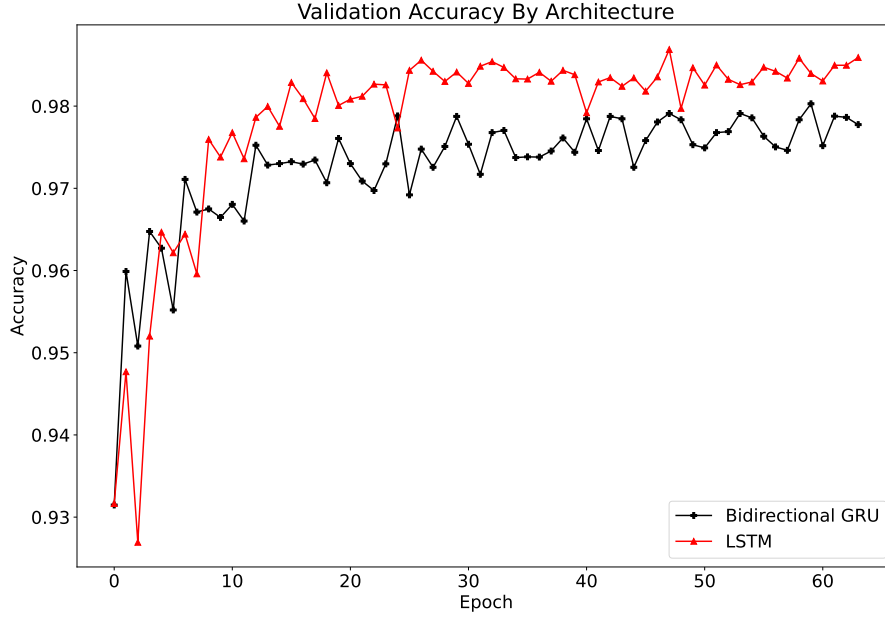


Figure 3. The validation accuracy for each proposed architecture.

compared to the bidirectional GRU which achieved 96.6% and 94.5% respectively for the same confidence levels.

This improvement at the higher levels is notable with respect to ensuring both safety and fairness. We have previously identified that it is paramount that systems detecting behaviour for the purposes of disciplinary actions must be able to minimize miss-classifications as much as possible. Especially with respect to fairness, false positives would punish non-aimbotters unjustly, a result that would likely lead to an erosion of trust in one’s player-base. Further, any server operator who provides player support whether through live chat, email, or phone would incur increased costs as a result of players likely attempting to appeal punishments related to false positives. Thus, our high confidence level is critical, since it raises the bar to an extremely high threshold to be classified as a positive detection. Further, it is important that this high confidence level does not result in too many false-negatives, as this would allow many aimbotters to evade detection. In these respects we believe our LSTM network’s improvements over the bidirectional GRU shows the LSTM network would provide server operators with a powerful tool for combating aimbot cheating. Though we want to stress that further testing of accuracy in a live scenario should be conducted for safety reasons before using this model to make punitive decisions on real players.

5. Practical Viability

5.1. Inference Speed

Our lab computer setup used an NVIDIA GeForce 1660Ti with 6GB of VRAM, a Ryzen7 2700X Eight-Core CPU, and 47GB of DDR4 RAM. With respect to our proposed LSTM network, we were able to achieve average inference speeds of 8.04 milliseconds per sample

over ten samples on the GPU where the GPU was given one warm-up sample prior to taking the average. We further achieved an average inference speed of 131.74 milliseconds per sample over ten samples on the CPU. These speeds are reasonable but problematic with respect to real-time monitoring as the server resources required to perform continual inference at these speeds on a large player base would be immense.

However, the speeds are low enough to use this model periodically. Some recommendations of areas where this model could be applied assuming the server captures and stores periodic or rolling movement samples of the correct inference sequence length:

- Upon a weak real time statistic flagging a player as suspicious.
- Upon receiving a manual user report for cheating.
- Upon a player winning a game, a sample of their movement data from the game is analyzed.
- Upon a player killing another player in game.

5.2. Framework Integration

It is clear that a server-side approach for aimbot detection is critical for Minecraft. With the game’s expansive modding community, any client-side logic would be easily defeated. A hacked client would simply remove any anti-cheat logic. Thus, for this aimbot detection system to be viable it must be integratable into a real Minecraft server. For the purposes of this study we briefly investigated a popular Minecraft server software. PaperMC is a fast and secure Minecraft server that also has a powerful plugin system for developers to add their own server functionality [18]. We must be able to use the plugin API to capture the player information needed to create equivalent time series to the ones used to train the model. The first required features `yaw`, `pitch`, `delta_yaw`, `delta_pitch`, `accel_yaw`, and `accel_pitch` are obtainable via the functions `getBodyYaw` from the `LivingEntity` class and `getPitch` from the `Entity` class which are both parents to the `Player` class and thus inherited by it [19, 20]. The next three features `target_x`, `target_y`, and `target_z` are obtainable via the function `getTargetBlock` from the `LivingEntity` class which is a parent to the `Player` class and thus inherited by it [19, 20]. Lastly, `position_x`, `position_y`, and `position_z` are all obtainable via the `getLocation` function from the `Entity` class which is a parent to the `Player` class and thus inherited by it [19, 21]. From our brief inspection of the plugin API, the information required to make use of our model in a live server is available.

6. Conclusion

This paper explored the growing issue of online cheating and its effects on game servers. We specifically turned our focus to aimbotting, which plagues first-person style games, where some form of combat and competition is involved. We identified a gap in any literature addressing the server-side detection of Minecraft aimbotting. We provided a background on aimbot detection techniques attempted by other authors in other online games. After some discussion on viable networks, we landed on the popular recurrent neural network architecture Long Short-Term Memory to provide a solution via classification on user movement time series data, hypothesizing that this network would outperform similar variants. Of particular interest was the bidirectional GRU network used in [6], where the authors saw strong performance on their specific dataset. We used the bidirectional GRU network on our dataset to provide our LSTM model a comparison to current literature. We identified that an aimbot detector would require very high confidence in its predictions in order to be considered fair and viable in real-world deployments. Our network provides noticeable improvements in accuracy at the required high confidence level, which provides both server

operators and players reassurance that the network is unlikely to falsely flag players for cheating. The LSTM outperformed the bidirectional GRU as shown in Table 2, with 98.5% accuracy at 95% confidence and 97.6% accuracy at 99% confidence. Further, we obtain better performance than other authors attempts at using an LSTM to detect aimbotting, although they used a different dataset [9, 10]. Lastly, we briefly explored the feasibility of this network being deployed in a real server scenario and concluded that this is possible as the accuracy is high enough to be considered reliable, feature data is collectible, and the network inference speed is reasonable.

7. Future Works

7.1. Wider Hyper-parameter Search

We only explored a limited range of hyper-parameters for the LSTM model. We believe there may be some merit to exploring other ranges of hyper-parameters especially with respect to larger sequence lengths. Though we were unable to achieve convergence at longer sequence lengths, there may be a better set of hyper-parameters that make this feasible.

7.2. Developing Server-side Integration

For the purposes of this paper we identified the feasibility of using this method on a live server. However, we did not attempt any real implementation of a plugin for some server to test our methods in the wild. We believe this may pose an interesting research topic with respect to monitoring real-time cheating.

Software and Data

The dataset used to conduct our study can be found on Kaggle [12]. The models used for this research were created using the PyTorch library: <https://pytorch.org/>.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. Any model whose goal is to flag suspicious behaviour for the purposes of punishing the offender should be scrutinized before being used in a live scenario. As such we advise caution when applying the methods discussed in this paper.

References

- [1] J. Jeff Yan and H. Choi. “Security issues in online games”. en. In: *The Electronic Library* 20.2 (Apr. 2002), pp. 125–133. ISSN: 0264-0473. DOI: [10.1108/02640470210424455](https://doi.org/10.1108/02640470210424455).
- [2] J. Yan and B. Randell. “A systematic classification of cheating in online games”. en. In: *Proceedings of 4th ACM SIGCOMM workshop on Network and system support for games*. Hawthorne NY: ACM, Oct. 2005, pp. 1–9. ISBN: 978-1-59593-156-6. DOI: [10.1145/1103599.1103606](https://doi.org/10.1145/1103599.1103606).
- [3] Statista. *Cumulative number of copies of Minecraft sold worldwide as of April 2023*. URL: <https://www.statista.com/statistics/680124/minecraft-unit-sales-worldwide/> (visited on 09/01/2024).
- [4] Krashnz. *Hypixel Network Server Stats*. URL: <https://hypixel.krashnz.com/> (visited on 09/27/2024).
- [5] CCBlueX. *LiquidBounce Website*. URL: <https://liquidbounce.net/> (visited on 09/01/2024).

- [6] M. Choi, G. Ko, and S. K. Cha. “BotScreen: Trust Everybody, but Cut the Aimbots Yourself”. In: *32nd USENIX Security Symposium (USENIX Security 23)*. Anaheim, CA: USENIX Association, Aug. 2023, pp. 481–498. ISBN: 978-1-939133-37-3. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/choi>.
- [7] Hsing-Kuo Pao, Kuan-Ta Chen, and Hong-Chung Chang. “Game Bot Detection via Avatar Trajectory Analysis”. In: *IEEE Transactions on Computational Intelligence and AI in Games* 2.3 (Sept. 2010), pp. 162–175. ISSN: 1943-068X, 1943-0698. DOI: [10.1109/TCIAIG.2010.2072506](https://doi.org/10.1109/TCIAIG.2010.2072506). (Visited on 11/15/2024).
- [8] D. Liu, X. Gao, M. Zhang, H. Wang, and A. Stavrou. “Detecting Passive Cheats in Online Games via Performance-Skillfulness Inconsistency”. In: *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. Denver, CO, USA: IEEE, June 2017, pp. 615–626. ISBN: 978-1-5386-0542-4. DOI: [10.1109/DSN.2017.20](https://doi.org/10.1109/DSN.2017.20).
- [9] H. Dunham. “Cheat Detection using Machine Learning within Counter-Strike: Global Offensive”. PhD thesis. The College of Wooster, 2020. URL: <https://openworks.wooster.edu/independentstudy/8948/>.
- [10] M. Willman. “Machine Learning to identify cheaters in online games”. PhD thesis. Umeå University, 2020. URL: <https://urn.kb.se/resolve?urn=urn:nbn:se:umu:diva-170973>.
- [11] J. P. Pinto, A. Pimenta, and P. Novais. “Deep learning and multivariate time series for cheat detection in video games”. en. In: *Machine Learning* 110.11-12 (Dec. 2021), pp. 3037–3057. ISSN: 0885-6125, 1573-0565. DOI: [10.1007/s10994-021-06055-x](https://doi.org/10.1007/s10994-021-06055-x).
- [12] FinalBoolean. *Aim Dataset for Minecraft*. URL: <https://www.kaggle.com/datasets/finalboolean/aim-dataset-for-minecraft> (visited on 09/01/2024).
- [13] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory”. en. In: *Neural Computation* 9.8 (Nov. 1997), pp. 1735–1780. ISSN: 0899-7667, 1530-888X. DOI: [10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).
- [14] K. Greff, R. K. Srivastava, J. Koutnik, B. R. Steunebrink, and J. Schmidhuber. “LSTM: A Search Space Odyssey”. In: *IEEE Transactions on Neural Networks and Learning Systems* 28.10 (Oct. 2017), pp. 2222–2232. ISSN: 2162-237X, 2162-2388. DOI: [10.1109/TNNLS.2016.2582924](https://doi.org/10.1109/TNNLS.2016.2582924).
- [15] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio. *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. Version Number: 1. 2014. DOI: [10.48550/ARXIV.1412.3555](https://doi.org/10.48550/ARXIV.1412.3555).
- [16] I. Sonata and Y. Heryadi. “Comparison of LSTM and Transformer for Time Series Data Forecasting”. In: *2024 7th International Conference on Informatics and Computational Sciences (ICICoS)*. Semarang, Indonesia: IEEE, July 2024, pp. 491–495. ISBN: 9798350375886. DOI: [10.1109/ICICoS62600.2024.10636892](https://doi.org/10.1109/ICICoS62600.2024.10636892).
- [17] Google. *Classification: Accuracy, recall, precision, and related metrics*. URL: <https://developers.google.com/machine-learning/crash-course/classification/accuracy-precision-recall> (visited on 09/01/2024).
- [18] PaperMC. *PaperMC Website*. URL: <https://papermc.io>.
- [19] Bukkit. *PaperMC Documentation - Player*. URL: <https://jd.papermc.io/paper/1.21.3/org/bukkit/entity/Player.html> (visited on 10/01/2024).
- [20] Bukkit. *PaperMC Documentation - LivingEntity*. URL: <https://jd.papermc.io/paper/1.21.3/org/bukkit/entity/LivingEntity.html> (visited on 10/01/2024).
- [21] Bukkit. *PaperMC Documentation - Entity*. URL: <https://jd.papermc.io/paper/1.21.3/org/bukkit/entity/Entity.html> (visited on 10/01/2024).